

Specification-First Model Orchestration Methodology

A token-efficient product development system for coordinating research, planning, validation, implementation, and escalation across multiple AI models.

Core principle

Do not spend expensive model tokens discovering the product while simultaneously building it. First create a validated specification; then use AI models primarily to execute that specification. Reserve stronger or paid models for tasks that genuinely require them.

Methodology name

Specification-First Model Orchestration (SFMO) is the recommended name. It describes both halves of the method: specification-first development and deliberate assignment of different AI models to different stages of the product lifecycle.

1. Objectives

- Minimize unnecessary AI token consumption.
- Separate product reasoning from code execution.
- Use free or low-cost models for research and routine work.
- Use paid/high-capability models only when their additional reasoning is valuable.
- Make application development reproducible across models and sessions.
- Reduce iterative architectural rewrites caused by vague requirements.

2. The SFMO Pipeline

The workflow is intentionally staged. Each stage produces an artifact that becomes input to the next stage.

RESEARCH → ARCHITECTURE → VALIDATION → SPECIFICATION FREEZE → IMPLEMENTATION → TESTING → REVIEW
→ ESCALATION

Research: gather facts, technologies, constraints, existing solutions, and relevant references.

Architecture: define the application before implementation: boundaries, modules, data flow, APIs, UI, database, security, and implementation order.

Validation: challenge the architecture using independent AI/model perspectives and external research.

Specification Freeze: stabilize the plan sufficiently that implementation can begin without repeatedly re-deciding foundational choices.

Implementation: assign coding tasks to the cheapest capable model first.

Escalation: move a task to a stronger model only when the current model is inefficient, stuck, or unsuitable.

3. Standard Project Documentation Architecture

```
PROJECT/  
■  
■ docs/  
■ ■■■ 00-project-overview.md  
■ ■■■ architecture.md  
■ ■■■ requirements.md  
■ ■■■ tech-stack.md  
■ ■■■ database.md  
■ ■■■ api.md  
■ ■■■ backend.md  
■ ■■■ frontend.md  
■ ■■■ ui-system.md  
■ ■■■ security.md  
■ ■■■ testing.md  
■ ■■■ deployment.md  
■ ■■■ decisions.md  
■ ■■■ implementation-plan.md  
■ ■■■ generation-guide.md  
■  
■ ■■■ backend/  
■ ■■■ frontend/  
■ ■■■ tests/  
■ ■■■ README.md
```

Document	Purpose
00-project-overview.md	Problem, goals, scope, users, constraints, success criteria.
architecture.md	System boundaries, components, data flow, dependencies, major design decisions.
requirements.md	Functional and non-functional requirements; acceptance criteria.
tech-stack.md	Languages, frameworks, libraries, versions, rationale, constraints.
database.md	Schema, relationships, indexes, migrations, data rules.
api.md	Endpoints, request/response contracts, errors, authentication, conventions.
backend.md	Backend modules, services, responsibilities, business logic boundaries.
frontend.md	Pages, components, state, routing, data flow, client behavior.
ui-system.md	Visual language, components, spacing, typography, responsive rules, accessibility.
security.md	Threat model, validation, authentication, authorization, secrets, secure defaults.
testing.md	Unit, integration, end-to-end, fixtures, acceptance and regression strategy.
deployment.md	Environment configuration, build/deploy process, observability and rollback.
decisions.md	Architecture Decision Records; why important choices were made.
implementation-plan.md	Ordered implementation phases, dependencies and completion criteria.
generation-guide.md	Rules AI agents must follow when creating or modifying code.

4. The Most Important Document: generation-guide.md

This file is the contract between the human-designed specification and coding agents. It should answer how an agent is allowed to modify the project, not merely what the project does.

Recommended sections

- Source-of-truth hierarchy: which document wins when two instructions conflict.
- Coding conventions and naming rules.
- Directory and file ownership rules.
- Dependency policy: when a new package may or may not be introduced.
- Architecture invariants that must never be silently changed.
- UI preservation rules.
- Database migration rules.
- Testing requirements for each implementation task.
- Definition of done.
- When the agent must stop and ask instead of guessing.

5. Specification Freeze

Before implementation begins, perform a deliberate freeze of foundational decisions. The freeze does not mean the project can never change. It means changes must be treated as new decisions rather than casual corrections.

Freeze checklist

- Requirements are explicit and testable.
- Architecture has been independently challenged.
- Technology choices have a documented reason.
- Database and API contracts are defined.
- Frontend and UI direction are sufficiently concrete.
- Implementation order and dependencies are known.
- Security and testing expectations exist before code generation.

6. Model Orchestration Strategy

Models should be selected by task capability and cost, not loyalty to one provider.

Stage	Preferred model class	Purpose
Research	Search/research model	Gather and validate external information.
Architecture	Strong reasoning model	Produce the initial blueprint and documentation.
Validation	Independent reasoning/search model	Challenge assumptions and identify omissions.
Routine coding	Free coding agent	Implement straightforward tasks.
Repository changes	Free/low-cost agent	Multi-file implementation and refactoring.
Hard debugging	Strong coding/reasoning model	Resolve difficult or persistent failures.
Final review	Independent strong model	Audit architecture, security, tests and quality.

7. Escalation Rules

Do not escalate simply because a model is slower. Escalate when the expected value of stronger reasoning exceeds its token cost.

- **Level 0:** local tools, autocomplete, manual edits, tests.
- **Level 1:** free coding agent for routine implementation.
- **Level 2:** second free agent for larger or alternative implementation tasks.
- **Level 3:** low-cost paid model for difficult execution.
- **Level 4:** premium model only for genuinely high-complexity reasoning or final audit.

Escalation trigger examples

- The same bug survives two credible attempts.
- The agent violates an architecture invariant.
- The task requires unfamiliar or subtle reasoning.
- A large refactor risks breaking multiple subsystems.
- Security, data integrity, or deployment correctness is at stake.

8. Implementation Task Format

Every coding task should be small enough to have a clear definition of done while containing enough context to prevent the agent from guessing.

Task: Implement X
Read: architecture.md, api.md, generation-guide.md
Relevant files: backend/auth/*
Constraints: do not change API contract; preserve existing UI
Acceptance criteria: ...
Tests required: ...
Stop conditions: ask before changing architecture or adding dependencies.

9. Change Control

When implementation reveals a flaw in the specification, do not immediately patch around it. Classify the change first.

Change type	Action
Minor implementation detail	Modify code; update relevant documentation if needed.
Local design improvement	Record the decision; implement and test.
API/database contract change	Stop, update specification, validate impact, then implement.
Architecture change	Return to architecture review; do not silently drift.
Requirement change	Update requirements and implementation plan before coding.

10. Token-Efficiency Principles

- **Plan once, execute many times:** invest human/model effort in a reusable specification.
- **Keep context modular:** load only the documents relevant to the current task when possible.
- **Exploit caching:** stable project instructions and documentation can benefit from provider prompt caching when supported.
- **Prefer patches over rewrites:** ask agents to modify only the necessary files.
- **Test locally:** a test result is cheaper and more reliable than repeated speculative prompting.
- **Don't regenerate finished work:** treat completed modules as stable unless a real requirement changes.
- **Escalate selectively:** reserve premium tokens for tasks where cheaper models demonstrably fail.

11. The Human's Role

SFMO is not an attempt to remove the programmer from the development process. The human remains responsible for product direction, constraints, architectural judgment, validation, acceptance, and final ownership.

The AI models become specialized collaborators: researcher, architect, critic, implementer, debugger, and auditor. This division of labor is what makes the system both cheaper and more controllable.

12. Definition of Success

A successful SFMO project is not one that uses the fewest tokens. It is one that produces a correct, maintainable application while minimizing wasted reasoning, repeated context, unnecessary rewrites, and uncontrolled architectural drift.

Short form: Research deeply → Specify completely → Validate independently → Freeze foundations → Execute cheaply → Escalate intelligently → Review rigorously.